

Joint Optimization of Caching, Computing, and Trajectory Planning in Aerial Mobile Edge Computing Networks: An MADDPG Approach

Haifeng Sun[✉], Yuqiang Zhou[✉], Hui Zhang[✉], Laha Ale[✉], *Member, IEEE*,
Hongning Dai, *Senior Member, IEEE*, and Ning Zhang[✉], *Senior Member, IEEE*

Abstract—The 6G network is expected to accommodate a wide array of connected devices, supporting diverse services from any location at any time. In this article, we introduce an aerial mobile edge computing (MEC) framework composed of high-altitude platforms (HAPs) and low-altitude unmanned aerial vehicles (UAVs), to cater to computing offloading for Internet of Things (IoT) devices, particularly in rural/remote areas or disaster zones. The framework accommodates various types of tasks, each computed by the corresponding Docker container. The objective is to achieve optimal workload fairness for UAVs while simultaneously minimizing the weighted processing costs among IoT devices in terms of task computation latency and energy consumption over the long term. This is achieved by jointly optimizing the flight trajectories and Docker image caching decisions of the UAVs with limited storage capacities, alongside ensuring service fairness for IoT devices. We tailor a multiagent deep deterministic policy gradient (MADDPG)-based approach to solve the long-term joint optimization problem, normalizing continuous actions and sampling discrete actions by generalizing the Gumbel-Softmax reparameterization trick. Experimental results indicate that our approach significantly outperforms benchmark schemes in terms of processing delay, energy consumption, and fairness.

Index Terms—Aerial access networks (AANs), joint optimization, multiagent deep deterministic policy gradient (MADDPG), mobile edge computing (MEC), task offloading.

I. INTRODUCTION

WITH the widespread adoption of mobile Internet services and continuous advancements in the Internet of Things (IoT), the demand for communication bandwidth in mobile Internet has surged significantly. To meet the Quality of Service (QoS) requirements for end IoT devices,

the concept of mobile edge computing (MEC) has been introduced to augment resource-limited IoT devices [1]. MEC facilitates the offloading of computation-intensive tasks from IoT devices to nearby MEC servers, thereby enhancing computational speed and reducing energy consumption. However, terrestrial networks cover just 20% of the total land area, and rural/remote areas or disaster zones often lack service coverage. Deploying fixed wireless access points (APs) with MEC service in these areas poses challenges due to difficulties and costs, as well as the inability to dynamically replace faulty equipment.

The ultimate goal of the 6G network is to overcome these issues and provide service anytime, anywhere. As a crucial component of 6G, aerial access networks (AANs) featured high-altitude platforms (HAPs) alongside low-altitude unmanned-aerial-vehicles (UAVs) can collectively provide a robust and flexible MEC infrastructure for ground IoT devices. HAPs can cover terrestrial areas spanning 60 to 80 km in diameter, carry powerful payload equipment (e.g., MEC servers, communication suits), and sustain operations for a period of several months [2]. HAPs have the capability to function as reliable airborne base stations offering extensive coverage and powerful payload capabilities, bridging the coverage gap caused by the limited service range of UAVs with auxiliary computational offloading service. Meanwhile, UAVs, acting as flying base stations, can carry MEC servers integrated with communication suits, embedded processors, and storage units to assist ground networks with computational offloading service. UAVs have the ability to adaptively change their positions based on the evolving offloading service requirements from ground IoT devices. The hover stability and line-of-sight transmission capabilities of UAVs provide high-speed and low-latency remote wireless access and MEC service for ground IoT devices [3]. UAVs establish links with ground devices through air-to-ground (A2G) links for task offloading service and can also communicate with HAPs through air-to-air (A2A) links [4].

Furthermore, MEC servers usually cache pertinent Docker images to facilitate the swift execution of required Docker containers. Docker containers, which are running instances housing essential computational programs, play a crucial role in efficient MEC offloading service [5]. Thus, UAVs can integrate Docker container platforms and cache Docker images for fast response. With greater capacity, HAPs can

Manuscript received 5 August 2024; accepted 5 September 2024. Date of publication 10 September 2024; date of current version 6 December 2024. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62261051. (Corresponding author: Haifeng Sun.)

Haifeng Sun, Yuqiang Zhou, and Hui Zhang are with the School of Computer Science and Technology, Southwest University of Science and Technology, Mianyang 621010, China (e-mail: sunhaifeng@swust.edu.cn; zhoyuqiang@mails.swust.edu.cn; zhanghui@swust.edu.cn).

Laha Ale is with the School of Computing and Artificial Intelligence, Southwest Jiaotong University, Chengdu 611756, China (e-mail: laha.ale@ieee.org).

Hongning Dai is with the Department of Computer Science, Hong Kong Baptist University, Hong Kong, China (e-mail: henrydai@hkbu.edu.hk).

Ning Zhang is with the Department of Electrical and Computer Engineering, University of Windsor, Windsor, ON N9B 3P4, Canada (e-mail: ning.zhang@ieee.org).

Digital Object Identifier 10.1109/IJOT.2024.3456846

integrate with a Docker image registry, a centralized store holding Docker images required to run the Docker containers deployed on ground IoT devices, UAVs, and the HAP itself to execute the corresponding tasks. If the UAV does not have the corresponding Docker images for a certain task, it needs to download it from the HAP's Docker image registry, leading to an increase in processing delay. Given that image caching significantly boosts the overall efficiency and responsiveness of the MEC system, the decision to cache Docker images on UAVs contributes to enhancing the overall effectiveness of MEC service.

In this work, we aim to provide long-term optimized offloading service for ground IoT devices in aerial MEC networks integrating MEC and AANs. Our objective is to maximize workload fairness for UAVs while integrating the minimization of weighted processing cost among IoT devices regarding the latency of task computation and energy consumption. This is achieved by jointly optimizing the flight trajectories and caching decisions for Docker images of UAVs, while also making fairly optimized computational task offloading decisions for IoT devices to ensure that every IoT device receives fair service.

Given that the UAV's flight angles and speeds are continuous variables, together with the correlation between offloading decision and Docker image caching decisions involves integer variables, the proposed joint optimization problem is fundamentally a mixed-integer nonlinear programming (MINLP). To tackle the MINLP optimization problem, we initially transform it into a multiagent Markov decision process (MAMDP). Subsequently, we propose an approach based on multiagent deep deterministic policy gradient (MADDPG) [6], a multiagent deep reinforcement learning (MADRL) algorithm that enables multiple agents to learn policies maximizing their cumulative reward [7]. In this approach, an independent agent is implemented on each UAV. These agents interact with the environment supported by both the HAP and A2A links. The main contributions of this article are outlined as follows.

- 1) We presented an MEC framework supported by AANs, featuring an HAP and numerous low-altitude UAVs. Through the jointly optimization of the flight trajectories and the Docker image caching decisions of UAVs, as well as the fairly optimized computational task offloading decisions of IoT devices, we attained the optimal workload fairness for UAVs, while simultaneously minimizing the weighted processing costs among IoT devices by task computation latency and energy consumption over the long term.
- 2) We develop an MADDPG-based approach to solve the joint optimization problem. Initially, we decouple the high-dimensional offloading decisions from the problem and design a low-complexity greedy method to address it for each given state. We then formulate the problem into an MAMDP. Subsequently, we tailored MADDPG by normalizing continuous actions and sampling discrete actions by generalizing the Gumbel-Softmax reparameterization trick.
- 3) We conducted simulations to assess the effectiveness of the proposed approach by evaluating its performance

in comparison to three benchmarks, with the evaluation criteria, including processing delay, energy consumption of IoT devices, and the fair distribution of workload among UAVs.

To the best of our knowledge, this work is the first comprehensive study focused on the joint optimization of caching, computing, and trajectory planning within aerial MEC networks. Our objective is to maximize service fairness among IoT devices and workload fairness among UAVs, while concurrently minimizing the weighted processing costs associated with task computation latency and energy consumption for IoT devices over the long term. This is accomplished by optimizing the flight trajectories of UAVs, determining caching decisions for Docker images retrieved from the Docker image registry on the HAP, and ensuring fair task offloading service for each IoT device.

The remaining sections of this article are organized as follows. In Section II, an overview of relevant research is presented. Section III details the system model and outlines the associated problem formulation. The specific approach design is expounded upon in Section IV, and Section V furnishes the numerical results along with a performance assessment. Finally, Section VI concludes this article.

II. RELATED WORK

A. Computing Offloading Enhanced by Docker Image Caching

Combining computing offloading with Docker image caching reduces latency, saves bandwidth, and lowers energy consumption for image downloading, which is vital for IoT devices. Some literature has focused on leveraging the benefits of Docker image employment and caching schemes.

Mou et al. [8] modeled task scheduling and image caching to minimize tasks' communication and computation delay. Zhang et al. [9] formulated an optimization model for image caching and task offloading. Bi et al. [10] considered allowing the server to selectively cache programs for future reuse and formulated a problem to jointly optimize caching placement, offloading decisions, and system resource allocation. The literature [11] discussed program caching in UAVs downloaded from the AP, focusing on offloading tasks from IoT devices to UAVs.

All the introduced image or program caching works are designed for different computing offloading scenarios. Unlike these works, we present an aerial MEC framework supported by AANs, optimizing Docker image caching decisions in combination with UAV trajectory planning. This approach ensures service fairness for IoT devices and addresses the optimization problem over the long term.

B. UAV-Aided Computing Offloading Architecture With Trajectory Planning

In the research on task offloading frameworks supported by AANs for efficient and adaptive MEC service, Granelli et al. [12], Lakew et al. [13], Jia et al. [14], and Kang et al. [15] proposed MEC frameworks comprising UAVs and HAPs to offer wireless access and computational

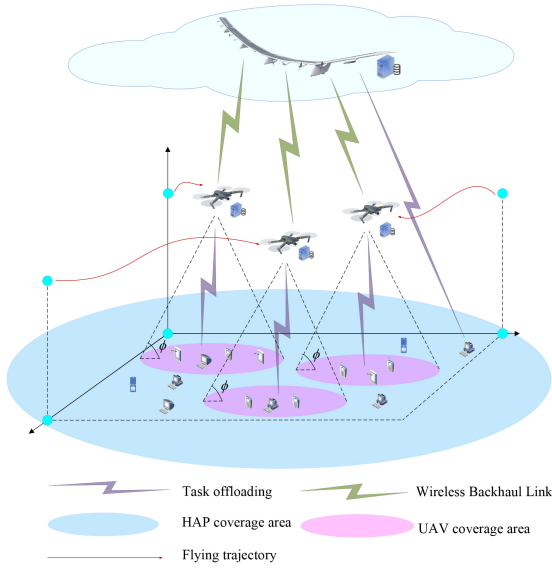


Fig. 1. System model.

task offloading for ground users. However, the frameworks only consider static UAVs. The authors optimized multi-UAV trajectory planning in [16], but the framework does not consider HAPs. Ke et al. [17] presented a network composed of HAPs to provide massive access and edge computing service, ensuring efficient connectivity and low latency for numerous IoT devices. However, the framework does not incorporate UAVs. More importantly, tasks generated in these proposed scenarios do not consider the requirements of different computing programs needed for task processing. Furthermore, these studies do not take into account the fairness of workload distribution among UAVs and fairness in device servicing. In reality, different tasks may correspond to different types of programs, and uneven UAV workloads can lead to resource wastage.

The aforementioned discussion on MEC task offloading has touched upon various studies concerning UAVs or HAPs. As far as we know, detailed collaborative models with Docker container platform supported computation and Docker image caching solutions for the cooperative provision of MEC service in hierarchical AANs are not fully developed.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. System Description

The designed MEC system model as shown in Fig. 1, integrating aerial MEC networks, comprises an HAP, numerous UAVs, and a large number of IoT devices. The IoT devices are represented as $n \in \mathcal{N} = \{1, 2, \dots, N\}$, randomly dispersed within a square region with each side measuring $l^{\max}$. Above this area, the UAVs are flying at a height of H , each identified by $m \in \mathcal{M} = \{1, 2, \dots, M\}$. Additionally, within this area, a fixed-position HAP, denoted as K , is hovering at an altitude of H_K ($H_K \gg H$). Although the HAP offers a Docker image registry, each UAV and the HAP itself are equipped with the edge servers that provide computational offloading services to ground IoT devices through Docker containers, which are instances created from Docker images.

For each time slot t in the discrete time system denoted by $t \in \mathcal{T} = \{1, 2, \dots, T\}$ with the time length τ , each resource constrained IoT device will generate a computation-intensive task, which can only be executed locally, or fully offloaded to either an UAV or the HAP for computing by the Docker container with the corresponding program running inside. Let $\mathcal{M}' = \{0, 1, 2, \dots, M + 1\}$ be a collection of possible locations where tasks can be executed. Therefore, the offloading decision variable for IoT device n in the t th time slot is given by the following equation:

$$\alpha_{n,m,t} = \{0, 1\}, m \in \mathcal{M}'. \quad (1)$$

For a task generated on IoT device n in the t th time slot, if it is optimized to be processed locally, then we have $\alpha_{n,0,t} = 1$, and $\alpha_{n,m,t} = 0$ for $m \neq 0$. If the task is offloaded to the HAP, we have $\alpha_{n,M+1,t} = 1$, thus $\alpha_{n,m,t} = 0$ for other values of m . When the task is offloaded to an UAV m , we have $\alpha_{n,m,t} = 1$, and $\alpha_{n,m,t} = 0$ for other values of m . Therefore, the decision variable for IoT device n at time slot t is specified as

$$\sum_{m \in \mathcal{M}'} \alpha_{n,m,t} = 1. \quad (2)$$

In each time slot, each IoT device is assigned a computation-intensive task $L_{n,t}$ for execution, which is expressed as

$$L_{n,t} = (C_{n,t}, D_{n,t}) \quad (3)$$

where $C_{n,t}$ is the amount of CPU cycles needed per bit and $D_{n,t}$ represents the input data size of the task.

Additionally, each task generated by an IoT device will be executed within a Docker container running the specific program and its dependencies required for processing the task. The set of Docker images stored in the HAP's Docker image registry is denoted as $\mathcal{J} = \{1, 2, 3, \dots, J\}$. We use $j = \psi_t(n) \quad \forall j \in \mathcal{J}$, to indicate that the task generated by IoT device n in time slot t requires the Docker image j to create a Docker container instance for execution. Significantly, different IoT devices may need the same Docker container to proceed their tasks on the same UAV. The HAP, equipped with a high-performance MEC server, stores all the Docker images in its Docker image registry required by IoT devices. However, UAVs have limited storage capacity and can only cache a portion of the Docker images. If the UAV does not cache the Docker image, it can request the image from the HAP and then download it through the wireless A2A link.

We assume that the caching probability of the Docker image depends on its popularity distribution of demanding, represented as F_i , $1 \leq i \leq J$, and the distribution of demands for all Docker images follows a Zipf distribution [18]. Therefore, the selection probability of F_i can be presented as follows:

$$F_i = \frac{i^{-\partial}}{\sum_{i=1}^J i^{-\partial}}, i \in \{1, 2, 3, \dots, J\} \quad (4)$$

where ∂ represents the degree of variation in Docker image demands, which varies based on the number of demands in each time slot, and a larger value indicates a greater difference in demands. This makes certain Docker images more likely to be cached, while others are relatively less likely to be selected. The denominator part $\sum_{i=1}^J i^{-\partial}$ is used to normalize the

probability distribution, ensuring that the sum of probabilities equals 1.

B. UAV Trajectory Control Model

In each time slot t , we assume that the HAP's position remains stable at $q_{K,t} = [X_{K,t}, Y_{K,t}, H_K]$, while for each UAV, its flight position is dictated by the flight angle $\theta_{m,t} \in [0, 2\pi)$ and the distance $d_{m,t} \in [0, d_{\max}]$. We set the initial coordinates of each UAV as $q_{m,0} = [X_{m,0}, Y_{m,0}, H]$. Therefore, the coordinate of an UAV at time slot t is derived as $q_{m,t} = [X_{m,t}, Y_{m,t}, H]$, where $X_{m,t} = X_{m,0} + \sum_{t'=1}^t d_{m,t'} \cos(\theta_{m,t'})$ and $Y_{m,t} = Y_{m,0} + \sum_{t'=1}^t d_{m,t'} \sin(\theta_{m,t'})$ [16]. Thus, we have

$$0 \leq X_{m,t} \leq l^{\max} \quad \forall m \in \mathcal{M} \quad (5)$$

and

$$0 \leq Y_{m,t} \leq l^{\max} \quad \forall m \in \mathcal{M}. \quad (6)$$

Additionally, the Euclidean distance among the two UAVs is expressed as

$$R_{m,m',t} = \|q_{m,t} - q_{m',t}\| \quad \forall m, m' \in \mathcal{M}, m \neq m'. \quad (7)$$

To avoid collisions during flights, UAVs should maintain a minimum distance of $R_{\min}$. Therefore, we have

$$R_{m,m',t} \geq R_{\min}. \quad (8)$$

Assuming the coordinates of the n th IoT device are given by $q_{n,t} = [X_{n,t}, Y_{n,t}, 0]$, the distance between IoT device n and UAV m at time slot t is defined as

$$R_{m,n,t} = \|q_{m,t} - q_{n,t}\| \quad \forall m \in \mathcal{M}. \quad (9)$$

At the same time, we define the distance between IoT device n and the HAP as

$$R_{K,n,t} = \|q_{K,t} - q_{n,t}\|. \quad (10)$$

The distance between the HAP and the low-altitude UAV is defined as

$$R_{K,m,t} = \|q_{K,t} - q_{m,t}\| \quad \forall m \in \mathcal{M}. \quad (11)$$

We assume that the service area of the HAP can cover all the UAVs and IoT devices in the proposed scenario, given its higher altitude and stronger communication capability. However, due to the limited service range of UAVs, only the IoT devices located within the coverage area of an UAV can receive offloading service. Following the approach in literature [19], we introduce ϕ as the minimum elevation angle of the UAV from an IoT device. Thus, we have

$$\alpha_{n,m,t} R_{m,n,t} \cos(\phi) \leq R_{\max} \quad \forall m \in \mathcal{M} \quad (12)$$

where $R_{\max}$ represents the maximum horizontal ground coverage radius of the UAV.

C. Communication Model

In line with prior research [20], we reasonably assume line-of-sight communication links between IoT devices and UAVs, as well as between IoT devices and the HAP. The channel power gains can, therefore, be derived based on distances from IoT devices to UAVs, according to the model of free-space path-loss [21]. Consequently, the channel power gain from IoT device n to an UAV m at time slot t follows the model:

$$g_{m,n,t} = g_0 R_{m,n,t}^{-2} = \frac{g_0}{\|q_{m,t} - q_{n,t}\|^2} \quad \forall m \in \mathcal{M} \quad (13)$$

where g_0 represents the channel power gain at a reference distance of 1 m.

To prevent transmission interference arising from multiple IoT devices potentially offloading tasks to the same UAV, we assume ground IoT devices can communicate with their respective UAVs using orthogonal frequency division multiple access (OFDMA) [22], [23]. Subsequently, the data rate available for the channel from IoT device n to UAV m can be computed as

$$r_{m,n,t} = B_n \log_2 \left(1 + \frac{P_n g_{m,n,t}}{\sigma_0^2} \right) \quad \forall m \in \mathcal{M} \quad (14)$$

where B_n represents the communication bandwidth between IoT device and the UAV, P_n denotes the transmission power of IoT device in the uplink link, and σ_0^2 represents the noise power.

Similar to the channel between IoT devices and UAVs, the channel power gain from IoT device n to the HAP at time slot t can be obtained as

$$g_{K,n,t} = g_0 R_{K,n,t}^{-2} = \frac{g_0}{\|q_{K,t} - q_{n,t}\|^2}. \quad (15)$$

Then, the wireless transmission rate from IoT device n to the HAP is denoted as

$$r_{K,n,t} = B_n \log_2 \left(1 + \frac{P_n g_{K,n,t}}{\sigma_0^2} \right). \quad (16)$$

Finally, as the HAP maintains stability at a relatively high altitude, similar to the low earth orbit (LEO) satellite channel, the fading effect arising from reflection and diffraction in the surrounding environments, such as hills, buildings, and trees, will be less pronounced compared to the ground IoT device channel with the UAV [24]. Following the Shannon theory [14], [25], the achievable data rate for the communication channel from the HAP to the UAV for Docker image downloading is calculated as:

$$r_{K,m,t} = B_K \log_2 \left(1 + \frac{P_K G_K L_{k,m} L_K}{K_B S_K B_K} \right) \quad \forall m \in \mathcal{M} \quad (17)$$

where B_K represents the channel bandwidth between the UAV and the HAP, G_K is the antenna power gain, P_K represents the transmit power from the HAP, L_K is the total path loss, K_B is the Boltzmann constant, S_K represents the system noise temperature, and $L_{k,m} = (c/4\pi R_{K,m} f_{K,m,n})^2$ is the free space loss, wherein $f_{K,m,n}$ is the central frequency.

D. Cache Model

Each UAV can cache certain Docker images used to create Docker container instances related to IoT device's tasks to save on wireless round-trip time of image downloading from the HAP [11]. We define $\beta_{m,j,t} \in \{0, 1\} \forall m \in \mathcal{M}, j \in \mathcal{J}, t \in \mathcal{T}$, to represent the discrete caching decision for the Docker image j at time slot t on UAV m . When $\beta_{m,j,t} = 1$, it signifies that the image j is cached on UAV m , otherwise, $\beta_{m,j,t} = 0$. Due to the limited cache capacity of each UAV, the following constraint applies:

$$\sum_{j \in \mathcal{J}} \beta_{m,j,t} z_j \leq Z_m \quad \forall m \in \mathcal{M} \quad (18)$$

where Z_m represents the total storage capacity of UAV m , and z_j denotes the size of the Docker image. Regarding the HAP, we assume it possesses ample storage capacity to cache all Docker images present in the Docker image registry.

E. Task Processing Model

Each IoT device has the option to process the task $L_{n,t}$ locally, offload it to an UAV, or remotely offload it to the HAP. The detailed information regarding task processing in different locations is listed below.

1) *Local Computation*: Assuming that each IoT device maintains a Docker container for task computation, the computation delay required to complete task $L_{n,t}$ is as follows:

$$T_{n,0,t} = \frac{C_{n,t} D_{n,t}}{f_n} \quad (19)$$

where f_n represents the operating clock frequency of IoT device n . Then, the needed energy consumption to perform this task is

$$E_{n,0,t} = k_n (f_n)^{v_n} T_{n,0,t} \quad (20)$$

where k_n is the effective switched capacitor, $v_n > 1$ [26].

2) *Offloading to UAV*: If the computation-intensive task $L_{n,t}$ is offloaded to be computed on UAV $m \forall m \in \mathcal{M}$, from the device n at time slot t , the delay mainly depends on whether the UAV has cached the corresponding Docker image or not.

If UAV m currently caches the required Docker image, the processing delay includes time for both data offloading and computation

$$T_{n,m,t}^{ca} = \frac{D_{n,t}}{r_{m,n,t}} + \frac{C_{n,t} D_{n,t}}{f_m} \quad (21)$$

where f_m is the clock frequency of the UAV. Due to the small size of the output data, task retrieval time is not considered.

While UAV m does not currently have the required Docker image cached for the task $L_{n,t}$, the processing delay should include the time needed for image downloading as

$$T_{n,m,t} = T_{n,m,t}^{ca} + \sum_{j \in \mathcal{J}} \mathbb{L}(\psi_t(n) = j) (1 - \beta_{m,j,t}) \frac{z_j}{r_{K,m,t}} \quad (22)$$

where $\mathbb{L}(\cdot)$ is the indicator function.

The corresponding offloading energy consumption for IoT device n is

$$E_{n,m,t} = P_n \frac{D_{n,t}}{r_{m,n,t}}. \quad (23)$$

3) *Offloading to HAP*: If the computation-intensive task $L_{n,t}$ is offloaded to be processed on the HAP at time slot t , the required processing delay includes time for both data offloading and computation on the HAP as

$$T_{n,M+1,t} = \frac{D_{n,t}}{r_{K,n,t}} + \frac{C_{n,t} D_{n,t}}{f_K}. \quad (24)$$

Similarly, the corresponding offloading energy consumption required by IoT device n is

$$E_{n,M+1,t} = P_n \frac{D_{n,t}}{r_{K,n,t}}. \quad (25)$$

So, the total processing delay and energy consumed by IoT device n to finish a task are, respectively, derived as

$$T_{n,m,t}^{\text{sum}} = \sum_{m \in \mathcal{M}'} \alpha_{n,m,t} T_{n,m,t} \quad (26)$$

and

$$E_{n,m,t}^{\text{sum}} = \sum_{m \in \mathcal{M}'} \alpha_{n,m,t} E_{n,m,t}. \quad (27)$$

Finally, we define the weighted processing cost of IoT device n as

$$\chi_{n,m,t} = \omega_T T_{n,m,t}^{\text{sum}} + \omega_E E_{n,m,t}^{\text{sum}} \quad \forall m \in \mathcal{M}' \quad (28)$$

where ω_T and ω_E are the weight factors for the delay and energy consumption of IoT devices in completing computation tasks, respectively, determined based on specific scenario requirements.

F. Workload Fairness Model

During UAV service, some UAVs may serve far more IoT devices than others, which leads to an unfair IoT workload distribution. A part of UAVs serving a majority of IoT devices bears a significantly higher IoT workload than the other part of UAVs serving fewer IoT devices. To solve this problem, we define $F_{m,t}$, $m \in \mathcal{M}$, as the quantity of IoT devices served by UAV m during time slot t . Thus, the workload fairness index f_t^{FU} , reflecting the workload fairness level among UAVs, can be defined as

$$f_t^{FU} = \frac{\left(\sum_{m=1}^M \sum_{t'=1}^t F_{m,t'} \right)^2}{M \sum_{m=1}^M \left(\sum_{t'=1}^t F_{m,t'} \right)^2}. \quad (29)$$

If all UAVs have a consistent IoT workload from the starting point until time slot t , the value of f_t^{FU} is approaching 1.

G. Problem Formulation

For the introduced framework, our objective is to maximize the workload fairness index f_t^{FU} of UAVs while simultaneously minimizing the sum of weighted processing cost of all IoT devices. The corresponding joint optimization problem for flight trajectories, caching policies of UAVs, and task offloading decision of IoT devices can be formulated as

$$(\mathbb{P}1) : \max_{\mathbf{A}, \mathbf{B}, \mathbf{C}} \sum_{t \in \mathcal{T}} \frac{f_t^{FU}}{\sum_{n \in \mathcal{N}} \sum_{m \in \mathcal{M}'} \chi_{n,m,t}} \quad (30a)$$

$$\text{s.t. } (1), (2), (5), (6), (8), (12), (18) \quad (30b)$$

$$0 \leq \theta_{m,t} < 2\pi \quad \forall m \in \mathcal{M}, t \in \mathcal{T} \quad (30c)$$

$$0 \leq d_{m,t} \leq d_{\max} \quad \forall m \in \mathcal{M}, t \in \mathcal{T} \quad (30d)$$

where $\mathbf{A} = \{\alpha_{n,m,t} \mid \forall m \in \mathcal{M}', t \in \mathcal{T}\}$, $\mathbf{B} = \{\beta_{m,j,t} \mid \forall m \in \mathcal{M}, j \in \mathcal{J}, t \in \mathcal{T}\}$, and $\mathbf{C} = \{\theta_{m,t}, d_{m,t} \mid \forall m \in \mathcal{M}, t \in \mathcal{T}\}$.

Due to the mixed nature of decision variables, and the high decision dimensions of IoT devices in the optimization problem, it is challenging to solve this MINLP problem using mathematical optimization methods. Additionally, directly applying model-free methods poses challenges because of the high-dimensional decision space in the MINLP problem. Hence, we devised an approach that addresses this issue by separating the offloading decisions of IoT devices from the trajectory planning and caching decisions of UAVs.

IV. APPROACH DESIGN

For the proposed MEC offloading framework in aerial MEC networks, considering the limited computing capacity of IoT devices, which are not suitable as intelligent agents for handling tasks involving MADRL, we treat the UAVs as intelligent agents.

By decoupling the high-dimensional offloading decisions $\mathbf{A} = \{\alpha_{n,m,t} \mid \forall m \in \mathcal{M}', t \in \mathcal{T}\}$ from the long-term optimization problem $\mathbb{P}1$ to achieve better convergence of MADRL, we introduce a low-complexity greedy method to address it. This method incorporates the service fairness constraint to determine if each IoT device is fairly served at each given state of MADRL.

A. Offloading Decision

To avoid situations where some IoT devices are continuously being served, while others are not receiving service for a long time, the service provision to IoT devices should exhibit service fairness [27]. We define $F_{n,t} = \{0, 1\}$, to indicate whether the task generated by IoT device n in time slot t is served. If the task is served, its value is 1; otherwise, it is 0. We define the service fairness constraint f_t^{FI} of all IoT devices at time slot t as

$$\begin{aligned} f_t^{FI} &= \mu_n - \varphi \cdot \sigma_n \\ &= \frac{\sum_{n=1}^N \sum_{t'=1}^t F_{n,t'}}{N} \\ &\quad - \varphi \sqrt{\frac{\sum_{n=1}^N \left(\sum_{t'=1}^t F_{n,t'} - \frac{\sum_{n=1}^N (\sum_{t'=1}^t F_{n,t'})}{N} \right)^2}{N-1}} \end{aligned} \quad (31)$$

where μ_n and σ_n represent the mean and standard deviation of the number of service times for all IoT devices, respectively. We introduce a service fairness factor φ , where $0 \leq \varphi \leq (\mu_n/\sigma_n)$, controlling the aggressiveness of the service fairness approach. As φ approaches (μ_n/σ_n) , the service fairness constraint f_t^{FI} approaches zero, and no unfairly served IoT devices are identified.

Define $b_{n,t} = \{0, 1\}$ to indicate whether IoT device n is unfairly served at t . If its total service count $\sum_{t'=1}^t F_{n,t'}$ falls below f_t^{FI} , it is identified as unfairly served (i.e., $b_{n,t} = 1$); otherwise, $b_{n,t} = 0$. If an IoT device is identified as unfairly

Algorithm 1 Greedy Offloading Decision of Each IoT Device

Input: the computed constraint f_t^{FI} , and the accumulated total service count of the n -th device $\sum_{t'=1}^t F_{n,t'}$;

Output: Offloading decision $\mathbf{A}^* = \{\alpha_{n,m,t}^*, \forall m \in \mathcal{M}', t \in \mathcal{T}\}$;

```

1: if  $\sum_{t'=1}^t F_{n,t'} \geq f_t^{FI}$  then
2:    $b_{n,t} = 0$ ;
3: else
4:    $b_{n,t} = 1$ ;
5: end if
6: if  $b_{n,t} == 0$  then
7:   for  $m \in \mathcal{M}^*$  do
8:     if  $m == \arg \min_{m \in \mathcal{M}^*} \chi_{n,m,t}$  then
9:        $\alpha_{n,m,t} = 1$ ;
10:    else
11:       $\alpha_{n,m,t} = 0$ ;
12:    end if
13:  end for
14: else
15:   for  $m \in \mathcal{M}'$  do
16:     if  $m == \arg \min_{m \in \mathcal{M}'} \chi_{n,m,t}$  then
17:        $\alpha_{n,m,t} = 1$ ;
18:    else
19:       $\alpha_{n,m,t} = 0$ ;
20:    end if
21:  end for
22: end if

```

served, it can select to offload its task $L_{n,t}$ to the HAP for computational assistance.

We employ a low-complexity greedy method to fairly optimize IoT device offloading decision. Define $\mathcal{M}^*$ as the set of potential locations in $\mathcal{M}'$ that do not include the HAP. Consequently, for IoT device n at a given time slot t , the offloading decision is made based on Algorithm 1.

Specifically, when $b_{n,t}$ is 0, it indicates that IoT device n is fairly served, and can only choose the most suitable place for execution from the set $\mathcal{M}^*$. On the other hand, $b_{n,t}$ is 1 signifying an unfairly served device of n , the device can choose the most suitable location for execution from the set $\mathcal{M}'$.

B. UAV Trajectory and Caching Decision

In the context of this framework, it is important to consider the actions of the UAVs, encompassing their movements and Docker image caching decisions, can impact the system's environmental states. The weighted processing cost, as defined by (28), is thereby determined by the system's current environmental state and the UAV's collective behaviors. Furthermore, the previous states and actions collectively drive the agents to transition into new random states, eliciting instantaneous rewards. Consequently, the task offloading optimization problem is formulated as a MAMDP denoted as $(\mathcal{M}, \mathcal{S}, \mathcal{A}, \mathcal{P}, \{\mathcal{R}_m\}_{m \in \mathcal{M}})$, where $\mathcal{M}$ represents the set of agents, each controlling an UAV. These agents interact with a changing environment defined by a series of states $\mathcal{S} = \{s_t, t \in \mathcal{T}\}$ and a series of actions $\mathcal{A} = \{a_t, t \in \mathcal{T}\}$ to maximize long-term rewards $\mathcal{R}_m = \{r_{m,t}, t \in \mathcal{T}\}$ [28].

The state s_t consists of private observations $o_{m,t}$. In each time slot, each UAV takes its own action $a_{m,t}$ based on its

private observation $o_{m,t}$ and receives instantaneous rewards $r_{m,t}$ [29]. The joint actions of all UAVs lead to a global state transition from the current state s_t to the next state s_{t+1} , following the state transition probabilities $\mathcal{P}$.

To address the optimization problem of UAV trajectory planning and caching decisions, we first transform it into an MDP and define its components—observations, actions, and reward functions—for each agent at time slot t as follows.

- 1) *Observation $o_{m,t}$* : The observation at time slot t encompasses various crucial components describing the state relate to UAVs, IoT devices and the HAP, which is defined as $o_{m,t} = (L_{n,t}, \psi_t(n), R_{m,m',t}, R_{m,n,t}, R_{K,n,t}, \chi_{n,m,t}, f_t^{FU})$.
- 2) *Action $a_{m,t}$* : We define the action as $a_{m,t} = (\theta_{m,t}, d_{m,t}, \beta_{m,j,t})$, representing the UAV's flight direction, distance, and caching decision for the t th time slot.
- 3) *Reward $r_{m,t}$* : According to the joint optimization problem, the reward function is defined as follows:

$$r_{m,t} = \frac{f_t^{FU}}{\frac{1}{N} \sum_{n \in \mathcal{N}} \sum_{m \in \mathcal{M}'} \chi_{m,n,t}} - p_m \quad (32)$$

where p_m signifies the penalty imposed on UAV m in the event of its departure from the target area or collision with other UAVs.

In MAMDP, the state transition of the environment is contingent upon the joint states and actions of agents. Therefore, we define the system states as $s_t = \{o_{m,t} \mid \forall m \in \mathcal{M}\}$, and the actions as $a_t = \{a_{m,t} \mid \forall m \in \mathcal{M}\}$.

C. Reformulating Actions of MADDPG

While value-based DRL algorithms, such as deep Q-network (DQN) [30], excel in handling discrete action spaces by estimating Q-values for each action, policy-based algorithms like deep deterministic policy gradient (DDPG) [31] are more effective in continuous action spaces due to their use of deterministic policies. To handle continuous action space problems, value-based algorithms can approximate continuous action spaces with a discrete set, but establishing a good approximation usually requires a large number of discrete actions [32]. Conversely, policy-based algorithms are more applicable for outputting discrete actions by approximating categorical samples via the Gumbel-Softmax reparameterization trick from a continuous distribution [33].

We designed an MADDPG-based approach to solve the MAMDP problem with hybrid continuous-discrete actions, since policy-based MADDPG is well-suited for addressing complex continuous actions in multiagent environments, as demonstrated in [34]. To adapt the actors' output for the formulated MAMDP problem, we employ action normalization for the continuous actions and generalize the Gumbel-Softmax reparameterization trick [35] to sample discrete actions without replacement.

1) *Continuous Actions Normalization*: We normalize the continuous actions, including the UAV's flight direction and distance, to reduce the dimension of the action space to a range between $[0, 1]$, which helps eliminate the wide difference in features and facilitates convergence [36]. Let a denote a

general output action by the actor network, with the maximum value of $a_{\max}$ and the minimum value of $a_{\min}$. The normalized action of a can be represented as follows:

$$\hat{a} = \frac{a - a_{\min}}{a_{\max} - a_{\min}} \quad (33)$$

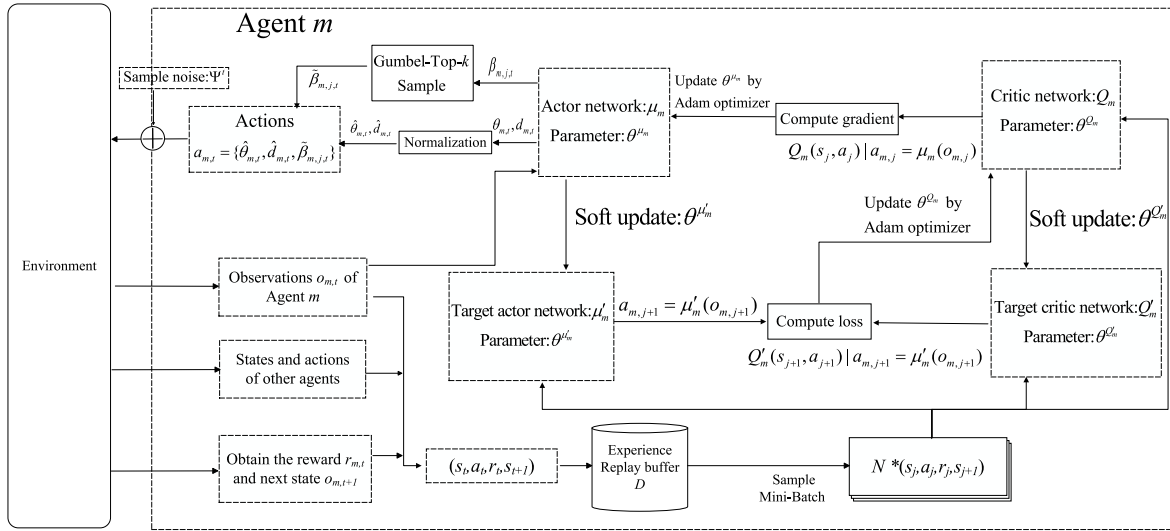
in which $a_{\min}$ and $a_{\max}$, representing the bounded flight direction of the UAV ($0 \leq \theta_{m,t} < 2\pi$) and distance ($0 \leq d_{m,t} \leq d_{\max}$), with $d_{\max}$ provided in Section V. The actions $\theta_{m,t}$ and $d_{m,t}$ can then be normalized as $\hat{\theta}_{m,t}$ and $\hat{d}_{m,t}$, respectively.

2) *Discrete Action Sampling*: In our model, each agent decides which Docker images to cache in each time slot. However, the Gumbel-Softmax reparameterization trick can only sample a single action corresponding to the highest probability value among the images. The Gumbel-top- k trick proposed in [35] provides a method to sample the top- k probability values without replacement. In contrast, we generalize the Gumbel-Softmax reparameterization trick at the actor network to continuously sample the top probability values of Docker images without replacement. After sampling the first action, we rerun the Gumbel-Softmax reparameterization trick with the remaining probabilities to sample the next action, ensuring compliance with the constraint defined in (18). We denote the sampled caching decision of $\beta_{m,j,t}$ as $\tilde{\beta}_{m,j,t}$.

D. MADDPG-Based Structure and Algorithm Design for Optimization Approach

Fig. 2 illustrates the tailored structure of the MADDPG agent m . Each agent is furnished with an actor network $\mu_m(s_t | \theta^{\mu_m})$, a critic network $Q_m(s_t, a_t | \theta^{Q_m})$, and their target networks $\mu'_m(s_t | \theta^{\mu'_m})$ and $Q'_m(s_t, a_t | \theta^{Q'_m})$, in which θ^{μ_m} , θ^{Q_m} , $\theta^{\mu'_m}$, and $\theta^{Q'_m}$ are the corresponding parameters of these four neural networks. Throughout the training process, each intelligent agent transmits its individual observations $o_{m,t}$ and actions $a_{m,t}$ to the environment built with the assistance of the HAP and A2A links. Subsequently, the environment sends back observations s_t and actions a_t to each respective agent. Then, each UAV can utilize the global state s_t and joint action a_t to train its critic network $Q_m(\cdot)$. The trained actor network $\mu_m(\cdot)$ of the agent is utilized to execute its policy, relying solely on its local state information $o_{m,t}$ to pick actions that maximize rewards, while the target network has the same network structure as $Q_m(\cdot)$, but with different parameters, used to estimate the value of future states for training stability.

In each time slot t , the actor network deterministically maps the local observation $o_{m,t}$ to a specific continuous action. To generate an exploratory policy, noise Ψ^t sampled from a Gaussian distribution with a mean of 0 and variance of 1 is added, resulting in $a_{m,t} = \mu_m(o_{m,t} | \theta^{\mu_m}) + \Psi^t$, where the exploration noise decays at a rate of 0.9995. The shared environment collects joint state s_t , joint actions a_t , joint reward r_t , and the next state s_{t+1} to each respective agent. To leverage experience for subsequent decision steps and enhance training sample efficiency and stability, agent m stores its current experience $e_t = \{s_t, a_t, r_t, s_{t+1}\}$ in an experience replay buffer D . For UAV m , to train the critic network and actor network, it randomly samples mini-batch of size S from the replay buffer D denoted as $\{s_j, a_j, r_j, s_{j+1}\}$.

Fig. 2. Structure of the tailored m th agent for MADDPG.

The loss function [6], which is utilized to train the critic network by minimizing the error in its predictions, is computed as

$$L(\theta^{Q_m}) = \mathbb{E} \left[\left(Q_m(s_j, a_j | \theta^{Q_m}) - r_{m,j} - \gamma Q'_m(s_{j+1}, a_{j+1} | \theta^{Q'_m}) \right)^2 \right]. \quad (34)$$

The expected reward gradient [6], which is sought to be maximized through the training of the actor network by adjusting its parameters for UAV m with a deterministic policy θ^{μ_m} is described as follows:

$$\nabla_{\theta^{\mu_m}} J = \mathbb{E} \left[\nabla_{\theta^{\mu_m}} \mu_m(o_{m,j} | \theta^{\mu_m}) \nabla_{a_{m,j}} Q_m(s_j, a_j | \theta^{Q_m}) \right]. \quad (35)$$

Each agent m updates the target networks as follows:

$$\begin{cases} \theta^{\mu'_m} \leftarrow \xi \theta^{\mu_m} + (1 - \xi) \theta^{\mu'_m} \\ \theta^{Q'_m} \leftarrow \xi \theta^{Q_m} + (1 - \xi) \theta^{Q'_m} \end{cases} \quad (36)$$

where ξ represents the update rate.

We provide pseudocode for the proposed approach in Algorithm 2. Specifically, we initialize between lines 1 and 4, where each UAV sets up its actor, critic, and two target networks. The training process then begins at line 6, where an UAV retrieves the observation result $o_{m,t}$ from the environment. Subsequently, in accordance with the obtained observation, each UAV selects an action $a_{m,t}$ generated by its actor network. The UAV then executes the action. Please note that if the UAV's next position goes outside the target area or collides with another UAV, it will remain in its current position and incur a penalty. Next, according to Algorithm 1, the IoT device selects the optimal UAV for Offloading. Following this, rewards $r_{m,t}$ and the next observation $o_{m,t+1}$ are obtained. Subsequently, each UAV stores this information in its experience replay buffer. From lines 28 to 33, when the learning process begins, random mini-batches are sampled from the experience replay buffer. Additionally, the critic network is updated through the loss function according to (34), the actor network is updated through the policy gradient

according to (35), and the target networks are updated through (36). Finally, the target networks are updated at a rate of τ .

V. PERFORMANCE EVALUATION

In this section, we demonstrate the effectiveness of the proposed approach based on MADDPG in the joint optimization problem through simulations. The simulation system is implemented using Python 3.9 and PyTorch 1.13.1. For the actor and critic networks, we employ three fully connected hidden layers with [300, 256, 128] neurons. The actor network is trained with a learning rate of 1×10^{-4} , and the critic network is trained with a learning rate of 1×10^{-3} . Adam optimizer is used to update the actor and critic networks [37].

A. Simulation Settings

We designate the target area as a square region measuring $1 \text{ km} \times 1 \text{ km}$, accommodating a maximum of 45 randomly distributed IoT devices. To manage tasks generated by these devices within the specified area, we deploy one HAP and four UAVs. The position of the HAP is fixed at [500 m, 500 m, 20 km], and the initial coordinates of the four UAVs are set to [100, 100, 200 m], [100, 900, 200 m], [900, 100, 200 m], and [900, 900, 200 m], respectively. We assume each Docker image has a uniform size of z , and each UAV has the same total storage capacity of Z . Then, the amount of cached images on the UAV is Z/z . Moreover, each IoT device generates one task per time slot. Additional critical simulation parameters are detailed in Table I.

To assess the performance of our developed MADDPG-based approach, we conducted a comparison with three benchmarks, all employing the IoT task offloading strategy described by Algorithm 1.

- 1) *Random*: For this approach, the UAV randomly selects the flight direction $\theta_{m,t} \in [0, 2\pi)$ and the flight distance $d_{m,t} \in [0, d_{\max}]$, with the constraint to fly within the designated area.

Algorithm 2 MADDPG-Based Optimization Approach for Integrating Workload Fairness and Processing Cost

```

1: for UAV  $m \in \mathcal{M}$  do
2:   Initialize the actor network  $\mu_m(s \mid \theta^{\mu_m})$  and the critic
     network  $Q_m(s, a \mid \theta^{Q_m})$ ;
3:   Initialize the target network  $\mu'_m(s \mid \theta^{\mu_m})$  and  $Q'_m(s, a \mid \theta^{Q_m})$ 
     with parameters  $\theta^{\mu'_m} = \theta^{\mu_m}$  and  $\theta^{Q'_m} = \theta^{Q_m}$ ;
4:   Initialize the experience replay buffer  $D$ ;
5: end for
6: for Episode = 1 to 17000 do
7:   for UAV  $m \in \mathcal{M}$  do
8:     Initialize observation  $o_{m,t}$ ;
9:   end for
10:  for  $t = 1$  to  $T$  do
11:    Obtain  $s_t$ ;
12:    for UAV  $m \in \mathcal{M}$  do
13:      Obtain action  $a_{m,t} = \mu_m(o_{m,t} \mid \theta^{\mu_m}) + \Psi^t$ ;
14:      Execute action  $a_{m,t}$ . If a UAV flies out of the target
         area or collides with another UAV, the UAV will
         remain in its current position.
15:    end for
16:    Obtain  $a_t$ ;
17:    Calculate  $f_t^{FI}$ ;
18:    for IoT devices  $n \in \mathcal{N}$  do
19:      Obtain the offloading decision  $\alpha_{n,m,t}$  according to
         algorithm 1;
20:      Calculate  $T_{n,m,t}^{sum}$  and  $E_{n,m,t}^{sum}$  according to Eq. (26)
         and Eq. (27), respectively;
21:    end for
22:    for UAV  $m \in \mathcal{M}$  do
23:      Obtain  $r_{m,t}$  according to Eq. (32);
24:      Obtain  $o_{m,t+1}$ ;
25:    end for
26:    Obtain  $s_{t+1}$ ;
27:    for UAV  $m \in \mathcal{M}$  do
28:      Store transition  $\{s_t, a_t, r_t, s_{t+1}\}$  into the experience
         replay buffer  $D$ ;
29:      if Training mode then
30:        Sample a random mini-batch of  $S$  transitions
          $\{s_j, a_j, r_j, s_{j+1}\}$  from  $D$ ;
31:        Update the critic network according to Eq. (34);
32:        Update the actor network according to Eq. (35);
33:        Update the target networks according to Eq. (36);
34:      end if
35:    end for
36:  end for
37: end for

```

- 2) *Circle*: All IoT devices are grouped into four clusters based on their coordinates, after which each UAV orbits the center of each cluster twice, maintaining a radius of $R^{\max}$.
- 3) *Proposed-No Cache*: To conduct a comprehensive analysis of the impacts of Docker image caching within the proposed MADDPG-based approach, this benchmark assumes that UAVs do not cache Docker images required for processing the offloaded tasks. This is achieved by consistently setting $\beta_{m,j,t} = 0$ in (22).

B. Simulation Results Analysis

To begin, we present the training curve of the proposed MADDPG-based approach in Fig. 3. It is evident that the initial cumulative reward remains below 100, gradually

TABLE I
SIMULATION PARAMETERS

Parameter	Value	Parameter	Value
N	$\{20, 25, \dots, 45\}$	B_n	10 MHz
l_{max}	1000 m	f_n	0.5 GHz
R_{max}	200 m	f_m	1.2 GHz
$C_{n,t}$	[1000,2000] cycles/bit	f_K	5 GHz
$D_{n,t}$	[10,15] Kb	T	24
K_B	1.38×10^{-23} J/K	σ_0^2	10^{-12} Watt
P_n	0.1 Watt	B_K	10 MHz
γ	0.95	ϕ	$\frac{\pi}{4}$
P_K	5 Watt	ξ	0.01
L_K	-80 dB	G_K	15 dB
R^{min}	20 m	S_K	1000 K
g_0	1.42×10^{-4}	J	5
z	6 Mbit	Z	18 Mbit
τ	1 s	d_{max}	50 m
k_n	10^{-28}	v_n	3

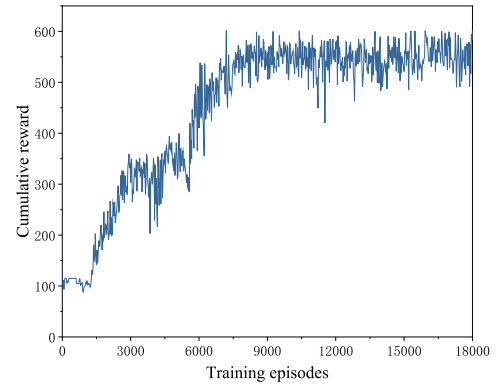


Fig. 3. Cumulative rewards across training episodes.

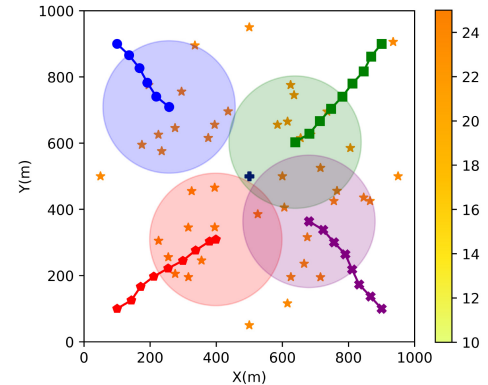


Fig. 4. Heat map of IoT devices and trajectories of UAVs ($\varphi = 0.5$).

increasing with the progression of training episodes. Around 6000 training episodes, the curve stabilizes, indicating convergence in the training process.

Fig. 4 illustrates both a heat map displaying the varying service times for each IoT device, with different color shadings inside the asterisks corresponding to the color bar, and the trajectories of UAVs generated from our proposed MADDPG-based approach using a service fairness factor of $\varphi = 0.5$. While the trajectories are depicted with dotted lines, the central cross indicates the location of the HAP. Analysis of the figure

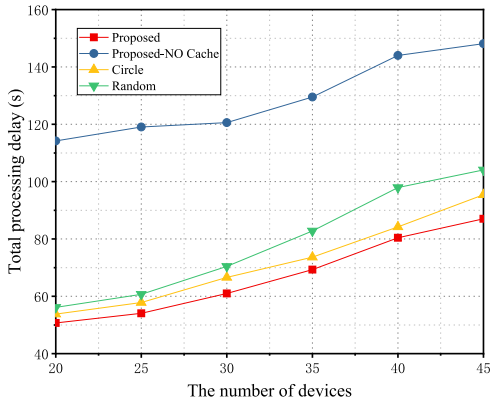


Fig. 5. Total task processing delay over 24 time slots across the number of IoT devices ($\varphi = 0.5$).

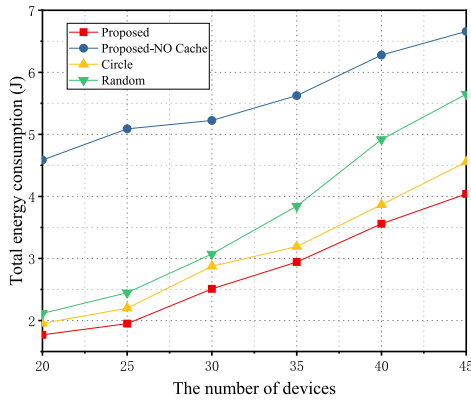


Fig. 6. Total energy consumption over 24 time slots across the number of IoT devices ($\varphi = 0.5$).

reveals that the UAVs effectively collaborate to cover the target areas as controlled by the proposed scheme. Once an UAV hits its optimal location, it hovers at this position. Remarkably, the color of the heat map indicates that IoT devices outside the service range of UAVs also receive fair service counts. This observation aligns with the concept that, through offloading decisions controlled by the service fairness factor f_t^{FI} , IoT devices can be fairly optimized to offload tasks to the HAP.

Figs. 5 and 6 depict the trends of total processing delay and energy consumption over 24 time slots, respectively, across varying numbers of IoT devices with a service fairness factor of $\varphi = 0.5$. As the number of IoT devices increases, both processing delay and energy consumption show noticeable growth. Among the strategies considered, the proposed MADDPG-based approach consistently outperforms the benchmark approaches with the least total processing delay and energy consumption, especially as the number of devices increases, while the proposed-no cache approach demonstrates the least favorable performance. This highlights the importance of implementing a caching strategy.

Fig. 7 illustrates the differences among the approaches in terms of the average UAV workload fairness f_t^{FU} , as defined by (29), across time slots, with 45 IoT devices and a service fairness factor of $\varphi = 0.5$. It is evident that the proposed MADDPG-based approach consistently outperforms

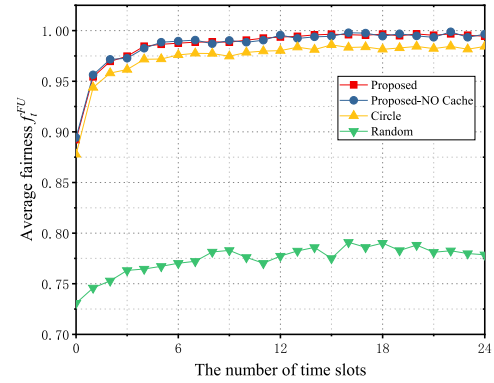


Fig. 7. Average UAV workload fairness (f_t^{FU}) for 45 IoT devices across time slots ($\varphi = 0.5$).

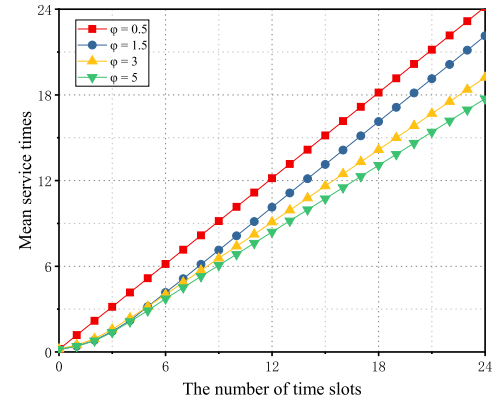


Fig. 8. Mean service times of the proposed MADDPG-based approach for 45 IoT devices across time slots at various values of φ .

the circle approach and the Random approach across all time slots. The proposed-no cache approach differs little from the proposed MADDPG-based approach because caching affect little for workload fairness. This superiority can be attributed to the inclusion of the workload fairness f_t^{FU} in the objective problem, effectively reducing workload variance among UAVs and enabling them to provide services in a more cooperative manner.

Fig. 8 illustrates the mean service times μ_n , as defined in (31), of the IoT devices within the proposed MADDPG-based approach for 45 IoT devices across time slots at various values of the service fairness factor φ . The mean service times exhibit the highest values when φ is set to 0.5. This phenomenon can be attributed to the inverse relationship between φ and the fairness threshold f_t^{FI} . As φ decreases, f_t^{FI} increases, resulting in more IoT devices being identified as unfairly served. Consequently, these devices are optimized for task offloading to the HAP.

VI. CONCLUSION

In this article, we explored the requirements of task offloading in specialized areas, such as emergency disaster zones and remote regions. Subsequently, we introduced an MEC task offloading framework in aerial MEC networks to fulfill the computing needs of IoT devices dispersed on the ground.

Unlike the existing frameworks, our approach capitalized on the mobility of UAVs and Docker image caching supported by the HAP to optimize workload fairness for UAVs while minimizing the total weighted processing cost of all IoT devices. To tackle this challenge, we proposed an approach that employed MADDPG to learn policies that maximize their cumulative reward within a given environment. Through simulations and comparison with the benchmark algorithms, we validated the effectiveness of our proposed approach. Results demonstrated that our approach achieved convergence after 6000 training iterations and effectively controlled UAV trajectories to cover a larger number of IoT devices. Moreover, our approach outperformed benchmark algorithms in terms of total processing delay, energy consumption, and equitable UAV workload fairness.

REFERENCES

- [1] Z. Na, C. Ji, B. Lin, and N. Zhang, "Joint optimization of trajectory and resource allocation in secure UAV relaying communications for Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 16284–16296, Sep. 2022.
- [2] M. S. Alam, G. K. Kurt, H. Yanikomeroglu, P. Zhu, and N. D. Dao, "High altitude platform station based super macro base station constellations," *IEEE Commun. Mag.*, vol. 59, no. 1, pp. 103–109, Jan. 2021.
- [3] C. Dong et al., "UAVs as an intelligent service: Boosting edge intelligence for air-ground integrated networks," *IEEE Netw.*, vol. 35, no. 4, pp. 167–175, Jul./Aug. 2021.
- [4] W. Zhang, L. Li, N. Zhang, T. Han, and S. Wang, "Air-ground integrated mobile edge networks: A survey," *IEEE Access*, vol. 8, pp. 125998–126018, 2020.
- [5] G. Avino, M. Malinverno, F. Malandrino, C. Casetti, and C. F. Chiasserini, "Characterizing docker overhead in mobile edge computing scenarios," in *Proc. Workshop Hot Topics Container Netw. Netw. Syst.*, 2017, pp. 30–35.
- [6] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 6379–6390.
- [7] W. Chen, X. Qiu, T. Cai, H.-N. Dai, Z. Zheng, and Y. Zhang, "Deep reinforcement learning for Internet of Things: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 3, pp. 1659–1692, 3rd Quart., 2021.
- [8] F. Mou, Z. Tang, J. Lou, J. Guo, W. Wang, and T. Wang, "Joint task scheduling and container image caching in edge computing," 2023, *arXiv:2310.00560*.
- [9] Q. Zhang, W. Xu, H. Luo, and S. Luo, "Fast globally optimal computational offloading and service caching in container-based edge computing systems," *IEEE Internet Things J.*, vol. 11, no. 13, pp. 23780–23792, Jul. 2024.
- [10] S. Bi, L. Huang, and Y.-J. A. Zhang, "Joint optimization of service caching placement and computation offloading in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 19, no. 7, pp. 4947–4963, Jul. 2020.
- [11] G. Zheng, C. Xu, M. Wen, and X. Zhao, "Service caching based aerial cooperative computing and resource allocation in multi-UAV enabled MEC systems," *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 10934–10947, Oct. 2022.
- [12] F. Granelli, C. Costa, J. Zhang, R. Bassoli, and F. H. P. Fitzek, "Design of an on-demand agile 5G multi-access edge computing platform using aerial vehicles," *IEEE Commun. Stand. Mag.*, vol. 4, no. 4, pp. 34–41, Dec. 2020.
- [13] D. S. Lakew, A.-T. Tran, N.-N. Dao, and S. Cho, "Intelligent offloading and resource allocation in heterogeneous aerial access IoT networks," *IEEE Internet Things J.*, vol. 10, no. 7, pp. 5704–5718, Apr. 2023.
- [14] Z. Jia, Q. Wu, C. Dong, C. Yuen, and Z. Han, "Hierarchical aerial computing for Internet of Things via cooperation of HAPs and UAVs," *IEEE Internet Things J.*, vol. 10, no. 7, pp. 5676–5688, Apr. 2023.
- [15] H. Kang, X. Chang, J. Mišić, V. B. Mišić, J. Fan, and Y. Liu, "Cooperative UAV resource allocation and task offloading in hierarchical aerial computing systems: A MAPPO based approach," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10497–10509, Jun. 2023.
- [16] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and L. Hanzo, "Multi-agent deep reinforcement learning-based trajectory planning for multi-UAV assisted mobile edge computing," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 1, pp. 73–84, Mar. 2021.
- [17] M. Ke et al., "An edge computing paradigm for massive IoT connectivity over high-altitude platform networks," *IEEE Wireless Commun.*, vol. 28, no. 5, pp. 102–109, Oct. 2021.
- [18] C. Li, Y. Zhang, X. Gao, and Y. Luo, "Energy-latency tradeoffs for edge caching and dynamic service migration based on DQN in mobile edge computing," *J. Parallel Distrib. Comput.*, vol. 166, pp. 15–31, Aug. 2022.
- [19] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and A. Nallanathan, "Deep reinforcement learning based dynamic trajectory control for UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 10, pp. 3536–3550, Oct. 2022.
- [20] Y. Zhou et al., "Secure communications for UAV-enabled mobile edge computing systems," *IEEE Trans. Commun.*, vol. 68, no. 1, pp. 376–388, Jan. 2020.
- [21] Y. Wang, W. Fang, Y. Ding, and N. Xiong, "Computation offloading optimization for UAV-assisted mobile edge computing: A deep deterministic policy gradient approach," *Wireless Netw.*, vol. 27, no. 4, pp. 2991–3006, 2021.
- [22] T. Ren et al., "Enabling efficient scheduling in large-scale UAV-assisted mobile-edge computing via hierarchical reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 10, pp. 7095–7109, May 2022.
- [23] S. Zhu, L. Gui, D. Zhao, N. Cheng, Q. Zhang, and X. Lang, "Learning-based computation offloading approaches in UAVs-assisted edge computing," *IEEE Trans. Veh. Technol.*, vol. 70, no. 1, pp. 928–944, Jan. 2021.
- [24] D. D. Mrema and S. Shimamoto, "Performance of quadrifilar helix antenna on EAD channel model for UAV to LEO satellite link," in *Proc. Int. Conf. Collaboration Technol. Syst. (CTS)*, 2012, pp. 170–175.
- [25] Z. Jia, M. Sheng, J. Li, D. Niyato, and Z. Han, "LEO-satellite-assisted UAV: Joint trajectory and data collection for Internet of Remote Things in 6G aerial access networks," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9814–9826, Jul. 2021.
- [26] H. Wang, H. Ke, and W. Sun, "Unmanned-aerial-vehicle-assisted computation offloading for mobile edge computing based on deep reinforcement learning," *IEEE Access*, vol. 8, pp. 180784–180798, 2020.
- [27] A. Mokhtari, M. A. Hossen, P. Jamshidi, and M. A. Salehi, "FELARE: Fair scheduling of machine learning tasks on heterogeneous edge systems," in *Proc. IEEE 15th Int. Conf. Cloud Comput. (CLOUD)*, 2022, pp. 459–468.
- [28] M. Elsayed, A. Badawy, A. E. Shafie, A. Mohamed, and T. Khattab, "A deep reinforcement learning framework for data compression in uplink NOMA-SWIPT systems," *IEEE Internet Things J.*, vol. 9, no. 14, pp. 11656–11674, Jul. 2022.
- [29] F. Ding, X. Zhang, and L. Xu, "The innovation algorithms for multivariable state-space models," *Int. J. Adapt. Control Signal Process.*, vol. 33, no. 11, pp. 1601–1618, 2019.
- [30] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [31] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2019, *arXiv:1509.02971*.
- [32] H. Fu, H. Tang, J. Hao, Z. Lei, Y. Chen, and C. Fan, "Deep multi-agent reinforcement learning with discrete-continuous hybrid action spaces," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 2329–2335.
- [33] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel-Softmax," in *Proc. 5th Int. Conf. Learn. Represent. (ICLR)*, 2017, pp. 1920–1931.
- [34] X. Zhang, Y. Zheng, L. Wang, A. Abdulali, and F. Iida, "Multi-agent collaborative target search based on the multi-agent deep deterministic policy gradient with emotional intrinsic motivation," *Appl. Sci.*, vol. 13, no. 21, pp. 11951–11957, 2023.
- [35] W. Kool, H. Van Hoof, and M. Welling, "Stochastic beams and where to find them: The Gumbel-top-k trick for sampling sequences without replacement," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 3499–3508.
- [36] L. Ale, N. Zhang, X. Fang, X. Chen, S. Wu, and L. Li, "Delay-aware and energy-efficient computation offloading in mobile-edge computing using deep reinforcement learning," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 3, pp. 881–892, Sep. 2021.
- [37] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.



Haifeng Sun received the Ph.D. degree with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China, in 2015.

From 2018 to 2019, he was a Visiting Scholar with the University of Waterloo, Waterloo, ON, Canada. He is currently an Associate Professor with the School of Computer Science and Technology, Southwest University of Science and Technology, Mianyang, China. His research interest includes resource allocation in mobile edge computing.



Yuqiang Zhou is currently pursuing the master's degree with the School of Computer Science and Technology, Southwest University of Science and Technology, Mianyang, China.

His research interests include resource allocation in mobile edge computing, UAV trajectory optimization, and deep reinforcement learning.



Hui Zhang received the Ph.D. degree from Japan Advanced Institute of Science and Technology, Nomi, Japan, in 2006.

He is currently a Professor and the Dean of the School of Computer Science and Technology, Southwest University of Science and Technology, Mianyang, China. His research focuses on data mining and knowledge engineering.

Dr. Zhang is the Deputy Director of the Natural Language Processing Committee of the Sichuan Computer Federation. He is also a Senior Member of the China Computer Federation.



Laha Ale (Member, IEEE) received the bachelor's degree in computer science from Southwest University of Science and Technology, Mianyang, China, in 2011, the M.B.A. degree from Webster University, Webster Groves, MO, USA, in 2016, and the Ph.D. degree in geospatial computer science from Texas A&M University at Corpus Christi, Corpus Christi, TX, USA, in 2021.

Before beginning his graduate program, he worked as a Software Engineer with Tieto, Beijing, China; Symantec, Chengdu, China; and Veritas, Chengdu, for seven years. In January 2022, he joined the Center for Computational Biomedicine, Harvard University, Cambridge, MA, USA, as a Postdoctoral Research Fellow. He joined the School of Computing and Artificial Intelligence, Southwest Jiaotong University, Chengdu, as an Associate Professor in 2023. His research interests include mobile edge computing, deep learning, deep reinforcement learning, deep universal probabilistic programming, and data science for biomedicine.



Hongning Dai (Senior Member, IEEE) received the Ph.D. degree in computer science and engineering from the Department of Computer Science and Engineering, Chinese University of Hong Kong, Hong Kong, and the D.Eng. degree in computer technology application from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China.

Before joining Hong Kong Baptist University, Hong Kong, he worked with the Department of Information Engineering, Chinese University of Hong Kong; the School of Computer Science and Engineering, Macau University of Science and Technology, Macau, China; and the Department of Computing and Decision Sciences, Lingnan University, Hong Kong. He has published more than 200 papers in referred journals and conferences, including Proceedings of the IEEE, IEEE JSAC, ACM CSUR, TPDS, IEEE INFOCOM, IEEE TRANSACTIONS ON COMPUTER, IEEE NETWORK, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, and IEEE COMMUNICATIONS SURVEYS AND TUTORIALS.

Dr. Dai has served as an Associate Editor for IEEE COMMUNICATIONS SURVEYS AND TUTORIALS, IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON INDUSTRIAL CYBER-PHYSICAL SYSTEMS, *Ad Hoc Networks* (Elsevier), and CONNECTION SCIENCE (Taylor and Francis). He is a Senior Member of ACM and EAI.



Ning Zhang (Senior Member, IEEE) received the Ph.D. degree in electrical and computer engineering from the University of Waterloo, Waterloo, ON, Canada.

He is currently an Associate Professor and the Canada Research Chair of the Department of Electrical and Computer Engineering, University of Windsor, Windsor, ON. His research interests include connected vehicles, mobile edge computing, wireless networking, and security.

Dr. Zhang is also a Distinguished Lecturer of IEEE ComSoc, a Highly Cited Researcher (Web of Science), and the Vice Chair of IEEE Technical Committee on Cognitive Networks and IEEE Technical Committee on Big Data. He serves/served as an Associate Editor for IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE COMMUNICATIONS SURVEYS AND TUTORIALS, IEEE INTERNET OF THINGS JOURNAL, and IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING. He also serves/served as the TPC/General Chair for numerous conferences and workshops, such as IEEE ICC, VTC, INFOCOM Workshop, and Mobicom Workshop.